DSC 140B

Representation Learning

Lecture 17 | Part 1

Transformers

Transformers

- Much of the recent progress in ML/AI is due to the **transformer** architecture.
- It is state of the art for many tasks.
  - Computer vision, natural language processing, speech recognition, etc.
- It's the "T" in "GPT".

Today

- The basics of the transformer architecture.

More Resources

- Peter Bloem, “Transformers from Scratch”. https://peterbloem.nl/blog/transformers
- Vaswani et al., “Attention Is All You Need” (2017). https://arxiv.org/abs/1706.03762
- Jay Alammar, “The Illustrated Transformer”. https://jalammar.github.io/illustrated-transformer/
- 3Blue1Brown, “Transformers, the tech behind LLMs” https://www.youtube.com/watch?v=wjZofJX0v4M

The Motivating Task

- **Sentiment analysis** is the task of determining the sentiment of a piece of text.

- Examples:
  - **Positive**: “I love this restaurant!”
  - **Negative**: “This restaurant is terrible.”

The Dataset

- We'll use a **synthetic** dataset of restaurant reviews.

- The reviews come in three "flavors":
  - "Simple"
  - "Reference"
  - "Override"

Example “Simple” Reviews

- “the location is amazing”
- “the food is horrible”
- “I believe the service is perfect”

Example “Reference” Reviews

- “The chef is disgusting. The music is wonderful. The former aspect was what really mattered.”
- “The waiter was terrible. It seemed to me that the bartender is superb. In the end the second thing was not what counted.”

Example “Override” Reviews

- “I believe the atmosphere was lousy. Yet more than anything the host was marvelous, in my opinion.”
- “Most importantly we thought that the host was mediocre. However in my opinion the drinks were fantastic.”
- “The ambiance was great in my experience. However the decisive point was that the food was marvelous.”

The Dataset

- Generated 10,000 training examples and 2,000 test examples.

- Rough breakdown:
  - Half “override” reviews.
  - 1/3 “reference” reviews.
  - 1/6 “simple” reviews.

DSC 140B

Representation Learning

Lecture 17 | Part 2

Approach 1: Bag of Words

Documents to Vectors

- Most ML methods accept vectors as input, but we have “documents”.
- In DSC 180, we learned a way to convert documents to vectors: **bag of words**.

Bag of Words

- First:
  1. Build a **dictionary** of all the words in the dataset.
  2. Choose an order for the words in the dictionary.

- Then, to convert a document to a vector:
  1. Initialize vector of zeros, same length as the dictionary.
  2. Set entry *i* to be number of times dictionary word *i* appears in the document.

Example

- Suppose our dictionary is:

```
["the", "food", "is", "amazing", "service", "horrible"]
```

- Then:
  - “the food is amazing” $\rightarrow (1, 1, 1, 1, 0, 0)$.
  - “the service is horrible” $\rightarrow (1, 0, 1, 0, 1, 1)$.
  - “the the amazing the” $\rightarrow (3, 0, 0, 1, 0, 0)$.

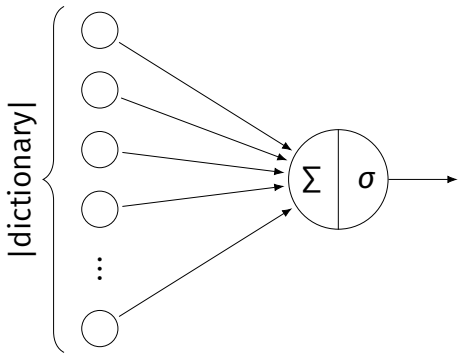
Observations

- Each document becomes a vector in $\mathbb{R}^{|\text{dictionary}|}$.
  - Potentially quite large.

- All information about word order is **lost**.
  - "the food is amazing" and "amazing the food is" have the same bag of words vector.

The Model

- We'll train a logistic regression model on the bag of words vectors.



$$H(\vec{x}) = \sigma(w_0 + w_1 x_1 + \ldots + w_d x_d)$$

Results

Review Type	Test Accuracy
Simple Reference Override	
Overall	

Results

Review Type	Test Accuracy
Simple Reference Override	**100%**
Overall	

Results

Review Type	Test Accuracy
Simple	100%
Reference	51.3%
Override	
Overall	

Results

Review Type	Test Accuracy
Simple	**100%**
Reference	**51.3%**
Override	**65.2%**
Overall	

Results

Review Type	Test Accuracy
Simple	**100%**
Reference	**51.3%**
Override	**65.2%**
Overall	66.1%

Takeaways

- BoW + logistic regression assigns a weight to each word in the dictionary.
  - Positive weight: associated with **positive** sentiment.
  - Negative weight: associated with **negative** sentiment.

- Prediction is weighted sum of words in document.

Takeaways

- This works well for simple reviews.
- Does not work when **word order matters**.

A fix?

- Instead of counting single words, we could count **bigrams** (pairs of consecutive words).
- Dictionary would contain entries like “the food”, “food is”, “is amazing”, etc.
- Embedding vector would count occurrences of “the food”, “food is”, etc.

A fix?

- Or, even better, we could use **trigrams**.
- Dictionary would contain entries like “the food is”, “food is amazing”, etc.
- Embedding vector would count occurrences of “the food is”, “food is amazing”, etc.

A fix?

- More generally, we could use n-grams for any n.

n-gram Results

Review Type	Unigram (BoW)	Bigram	Trigram	10-gram
Simple	**100%**	**100%**	**100%**	**100%**
Reference	**50.8%**	**51.3%**	**52.1%**	**57.6%**
Override	**65.2%**	**68.3%**	**70.3%**	**70.2%**
Overall	66.1%	67.8%	69.1%	70.9%

Observations

- *n*-grams help us learn some word order information.
- However, we would need big *n*-grams to capture long-distance dependencies.
  - "The waiter was terrible. The bartender is superb. In the end the second thing was not what counted."
- **Problem**: when *n* is large, most *n*-grams from the training set do not appear in the test set.

Moving Forward

- The bag of words representation was **too simple**.
- **Next:** learning a (better?) representation.

DSC 140B

Representation Learning

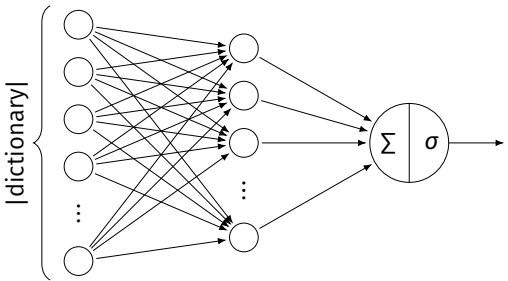
Lecture 17 | Part 3

Continuous Bag of Words

Idea

- Bag of words is **too simple** of a representation.
- Deep networks can **learn** good representations.
- Let's make our previous model *deep* by adding a hidden layer.

Architecture



- Input is bag of words vector.
- Hidden layer performs embedding.
- Dimensionality of embedding = number of hidden nodes.
- Use nonlinear activation function (e.g. ReLU) on hidden layer.

Architecture (cont.)

- We'll use 100 hidden nodes.
- Each word/document is embedded into $\mathbb{R}^{100}$.
- Embedding is learned while optimizing the model for the classification task.

Results

Review Type	BoW	CBoW
Simple	**100%**	
Reference	**50.8%**	
Override	**65.2%**	
Overall	66.1%	

Results

Review Type	BoW	CBoW
Simple	**100%**	**100%**
Reference	**50.8%**	
Override	**65.2%**	
Overall	66.1%	

Results

Review Type	BoW	CBoW
Simple	100%	100%
Reference	50.8%	49.3%
Override	65.2%	
Overall	66.1%	

Results

Review Type	BoW	CBoW
Simple	**100%**	**100%**
Reference	**50.8%**	**49.3%**
Override	**65.2%**	**66.2%**
Overall	66.1%	

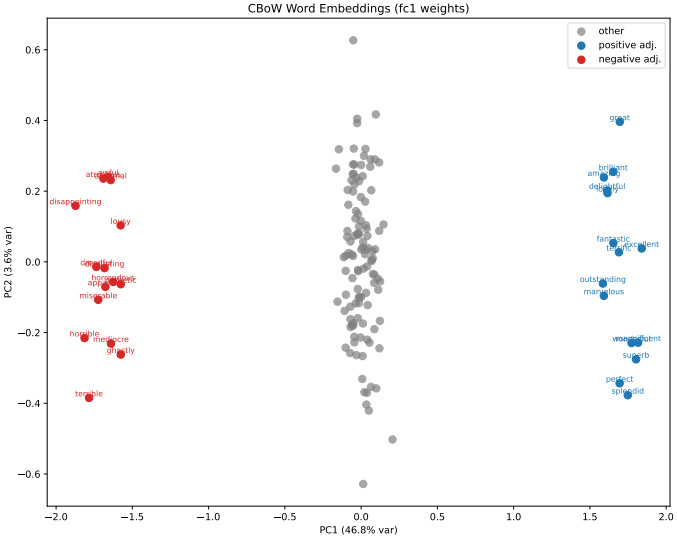
Results

Review Type	BoW	CBoW
Simple	**100%**	**100%**
Reference	**50.8%**	**49.3%**
Override	**65.2%**	**66.2%**
Overall	66.1%	66.1%

Visualizing the Embeddings

- Each word is embedded into $\mathbb{R}^{100}$.
- We can project these embeddings down to $\mathbb{R}^2$ using PCA.

Visualizing the Word Embeddings



Continuous Bag of Words

- This model is a **continuous bag of words** (CBoW) model.
- Each word is no longer a one-hot vector.
  - Now: a dense vector in $\mathbb{R}^{100}$.
- Each document is no longer a count vector.
  - Now: a weighted sum of word embeddings.

Where next?

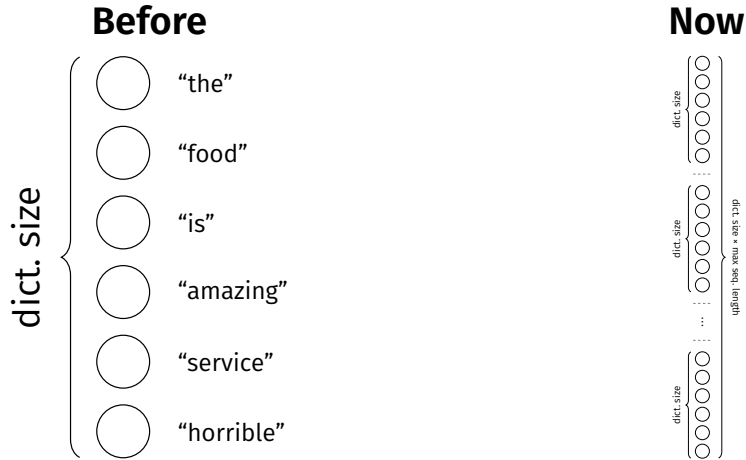
- CBoW gave no improvement over BoW.
- It still doesn't capture word order.
  - It never even had a chance to!
  - The input is still a bag of words vector, which doesn't capture word order.
- Let's try to preserve word order.

Sequences as Input

- **Before**: document was input as a single bag of words vector.
  - A vector in $\mathbb{R}^{|\text{dictionary}|}$.
- **Now**: document is input as a **sequence** of one-hot word vectors.

Sequences as Input

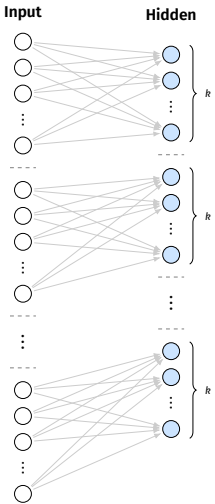
“The service is horrible”



Input Layer

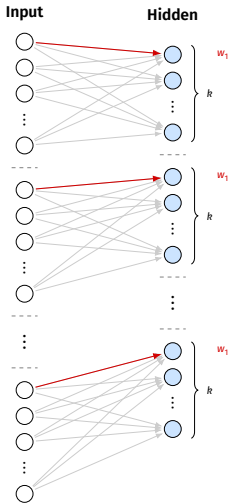
- There are now |dictionary| × max seq. length input nodes.
  - max. sequence length = max length of any document
- Many more than before!
- However, word order is now **preserved**.

Input → Hidden



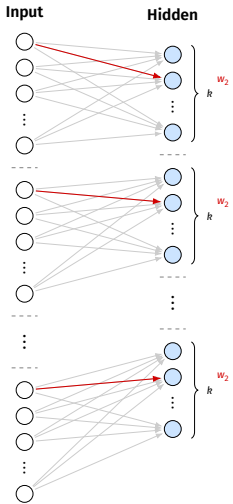
- Sequence of words is mapped to sequence of embeddings, each in $\mathbb{R}^k$.
- Weights are shared.

Input → Hidden



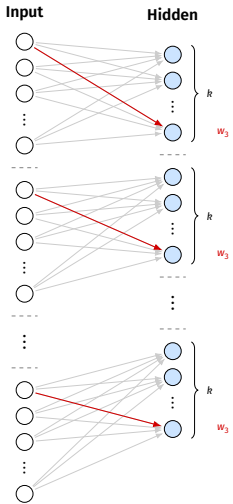
- Sequence of words is mapped to sequence of embeddings, each in $\mathbb{R}^k$.
- Weights are shared.

Input → Hidden



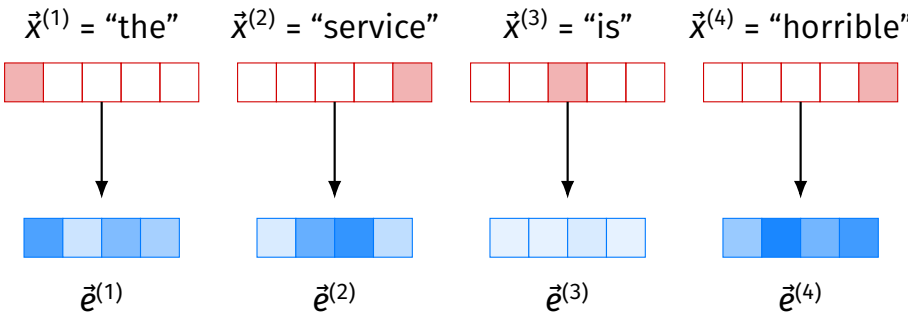
- Sequence of words is mapped to sequence of embeddings, each in $\mathbb{R}^k$.
- Weights are shared.

Input → Hidden



- Sequence of words is mapped to sequence of embeddings, each in $\mathbb{R}^k$.
- Weights are shared.

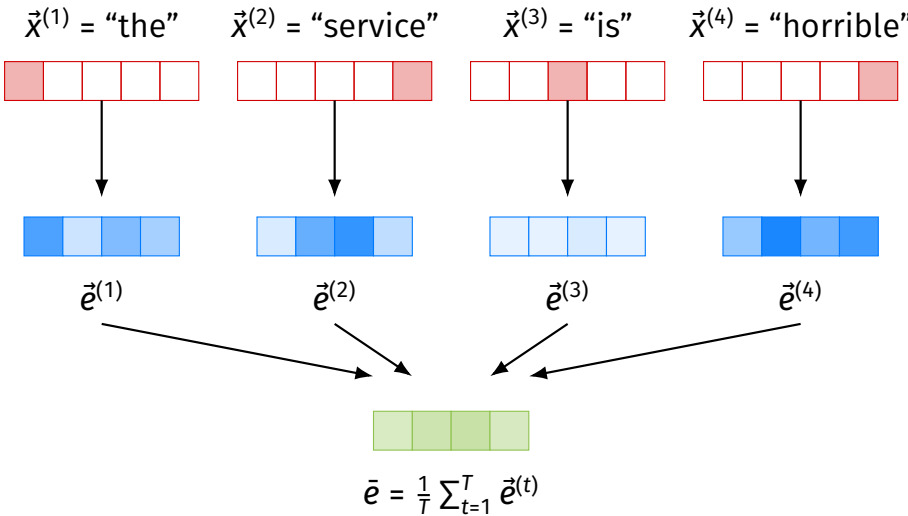
Input → Hidden



Pooling

- Now we have a sequence of embeddings, one for each word.
- Like in BoW, we add (average) these embeddings together to get single vector for the document.

Input → Hidden → Pooling



Output

- We feed the pooled vector into a fully-connected feed-forward network to make predictions.
- Embeddings are trained to be useful for the classification task.
- This model is called a **DAN** (Deep Averaging Network).

Problem

- Although input preserves word order, the pooling step still loses it.
- But it sets us up for the transformer.

DSC 140B

Representation Learning

Lecture 17 | Part 4

Attention

Importance of Context

- The meaning of words often depends on context and word order.

- Example:
  - “I walked up to the river and sat next to the bank.”
  - “After going to the post office I sat next to the bank.”

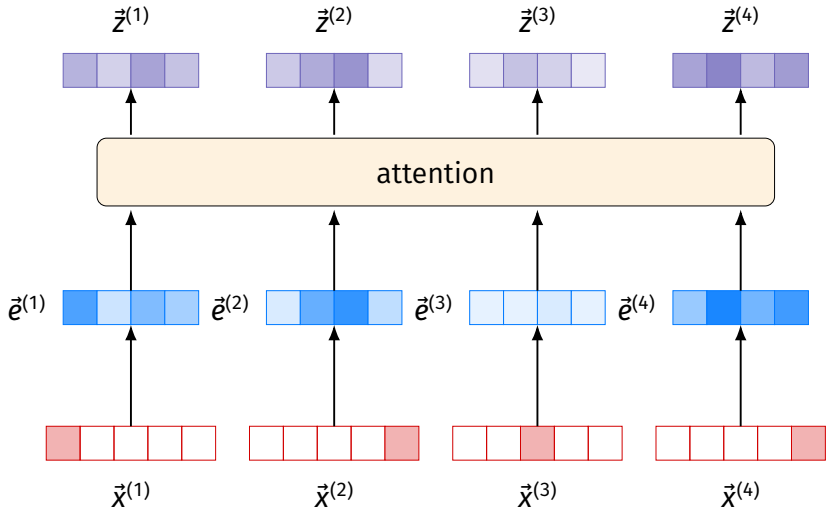
Idea

- When embedding, let each word “look” at other words in the sentence.
- Each word decides how much **attention** to pay to every other word.
- Example:
  - “I walked up to the river and sat next to the bank.”
  - “After going to the post office I sat next to the bank.”

Setup

- Suppose $\vec{e}^{(1)}, \vec{e}^{(2)}, \ldots, \vec{e}^{(T)}$ are embeddings for a sentence of length T.
- We want to create new embeddings $\vec{z}^{(1)}, \vec{z}^{(2)}, \ldots, \vec{z}^{(T)}$ that are context-aware.

Attention


$\vec{z}^{(1)}$
$\vec{z}^{(2)}$
$\vec{z}^{(3)}$
$\vec{z}^{(4)}$
attention
$\vec{e}^{(1)}$
$\vec{e}^{(2)}$
$\vec{e}^{(3)}$
$\vec{e}^{(4)}$
$\vec{x}^{(1)}$
$\vec{x}^{(2)}$
$\vec{x}^{(3)}$
$\vec{x}^{(4)}$

Setup

- Assume new embedding for word i is a weighted average:

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$

- Where:
  - $\text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)})$ is a function that computes how much attention word i should pay to word j.
  - $\vec{f}(\vec{e}^{(j)})$ transforms the embedding of word j.

How?

- We need to define Attention($\vec{e}^{(i)}, \vec{e}^{(j)}$) and $\vec{f}(\vec{e}^{(j)})$.
- How do we know how much attention word i should pay to word j?
- We don't! Let the network **learn** this.

Learning Attention Scores

- Define $\omega_{ij} = \left(W_q \vec{e}^{(i)}\right) \cdot \left(W_k \vec{e}^{(j)}\right)$, where W_q and W_k are **learnable** weight matrices.
- ω_{ij} is the **attention score** that word i pays to word j.
- Normalize attention scores using softmax:

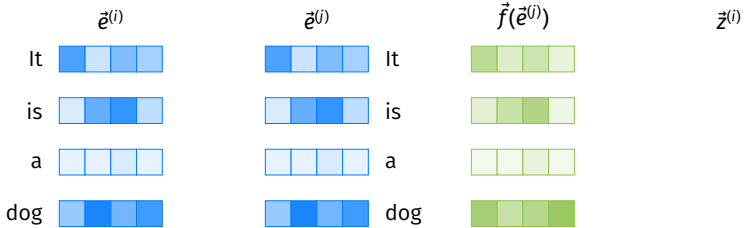
$$\text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) = \frac{\exp(\omega_{ij})}{\sum_{k=1}^{T} \exp(\omega_{ik})}$$

How?

- Also need to define $\vec{f}(\vec{e}^{(j)})$.
- Let $\vec{f}(\vec{e}^{(j)}) = W_v \vec{e}^{(j)}$, where W_v is another **learnable** weight matrix.

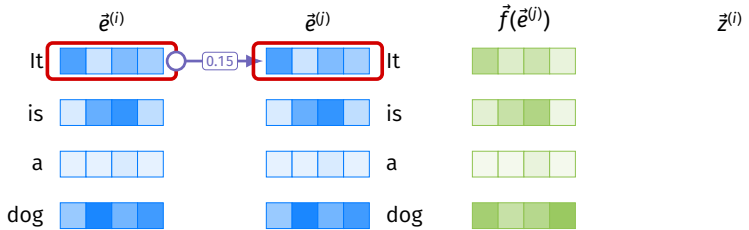
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



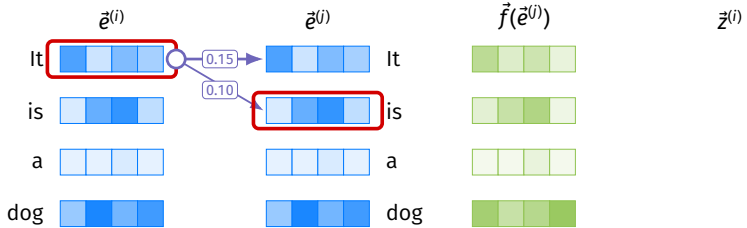
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



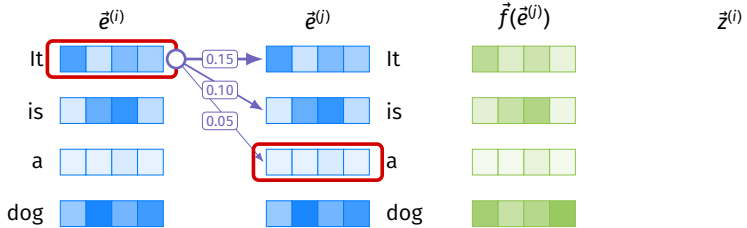
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



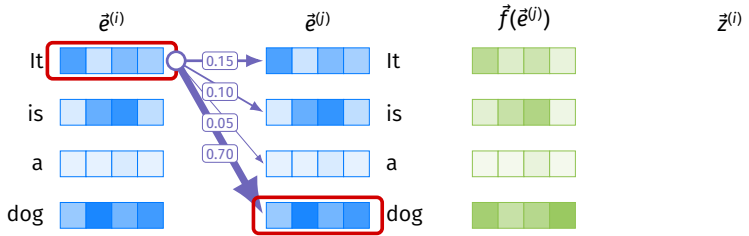
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



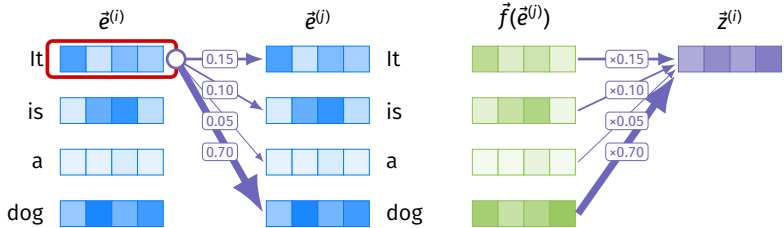
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



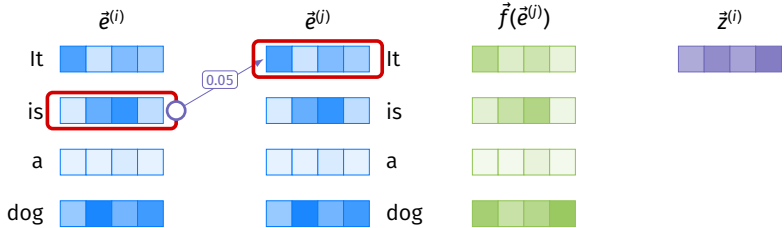
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



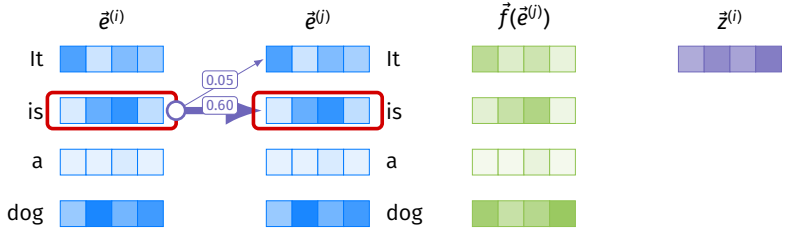
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



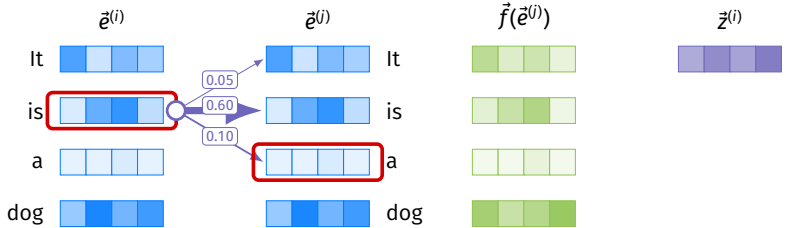
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



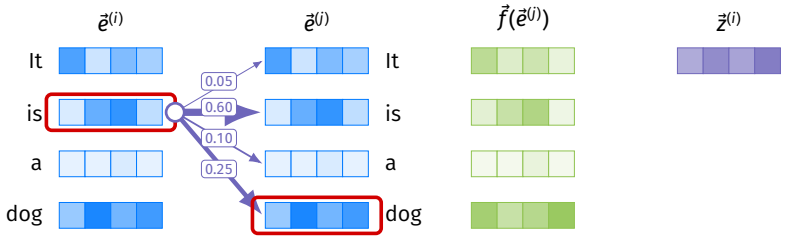
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



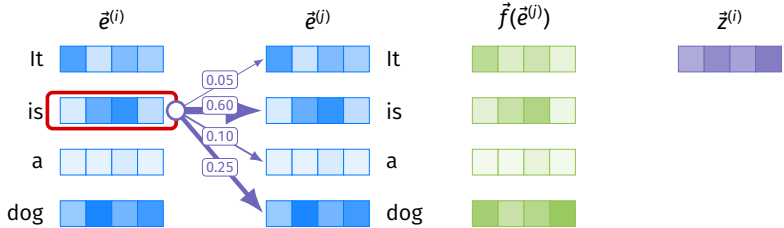
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



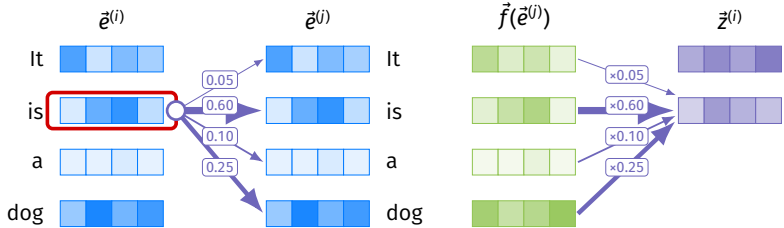
Example

$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$

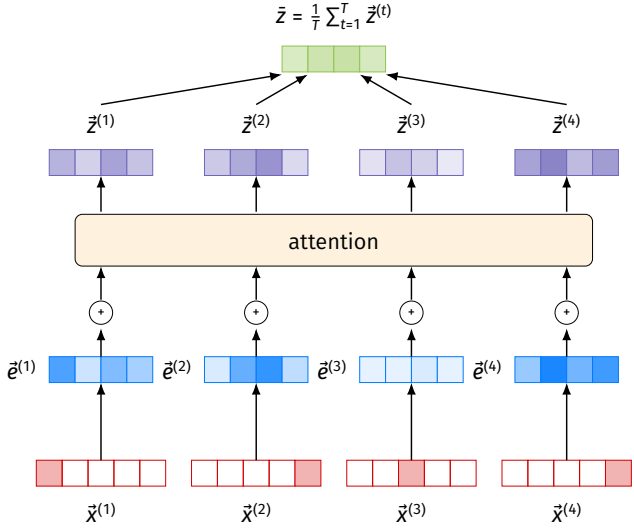


Example

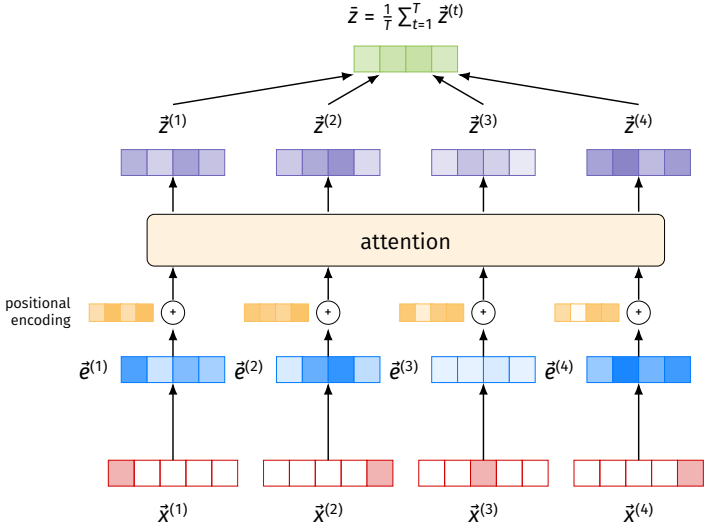
$$\vec{z}^{(i)} = \sum_{j=1}^{T} \text{Attention}(\vec{e}^{(i)}, \vec{e}^{(j)}) \times \vec{f}(\vec{e}^{(j)})$$



Architecture



Architecture



Architecture

- The pooled $\bar{z}$ is fed into a classification network to make predictions.

Training

- We'll train this network to make good predictions.
- In the process, it will **learn** good embeddings that make the Attention mechanism do meaningful things.

Results

Review Type	DAN	Attention
Simple	**100%**	
Reference	**49.9%**	
Override	**64.6%**	
Overall	65.3%	

Results

Review Type	DAN	Attention
Simple	**100%**	**100%**
Reference	**49.9%**	
Override	**64.6%**	
Overall	65.3%	

Results

Review Type	DAN	Attention
Simple	100%	100%
Reference	49.9%	92.6%
Override	64.6%	
Overall	65.3%	

Results

Review Type	DAN	Attention
Simple	100%	100%
Reference	49.9%	92.6%
Override	64.6%	94.4%
Overall	65.3%	

Results

Review Type	DAN	Attention
Simple	**100%**	**100%**
Reference	**49.9%**	**92.6%**
Override	**64.6%**	**94.4%**
Overall	65.3%	**94.8%**

Transformers

- A **transformer** is a network that uses attention as its main building block.
- This was a very simple transformer.
- In practice:
  - Use multiple attention **heads** in parallel.
  - Stack multiple layers of attention.
  - Use residual connections and layer normalization.

Text Prediction

- We attached a binary classification network to the end of our transformer.
- But we could also try to predict the next word in the sequence.
  - I.e., a multiclass classifier that outputs a distribution over the dictionary.
- This is the architecture behind LLMs.

DSC 140B

Representation Learning

Lecture 17 | Part 5

Where next?

Other classes

- **CSE 151B**: Deep Learning
- HDSI summer workshop on deep learning?

The End

- Keep in touch!